



Google worsens AMP

In an attempt to address one of the biggest criticisms of AMP, Google has invented Signed Exchanges. <https://digit.in/may19-5>



VSCode without trackers

An open sourced Visual Studio Code that doesn't pass data back to Microsoft has been released. <https://digit.in/may19-6>

```
def unpause(self):
    # This allows the player to still exit the game
    # paused, rather than having to unpause first.
    self.event_close()

    else:
        if self.pairs is None:
            self.unpause()

def event_paused_close(self):
    # This allows the player to still exit the game
    # rather than having to unpause first.
    self.event_close()
```

Additional methods to handle game events

resolution, the minimum generation time, the maximum and minimum number of invaders possible, etc. We also keep a method for a new generation in which at every new generation, the time between two consecutive generations are reduced, as a means to keep the game getting harder. As far as we are concerned with the events that shall occur at each `event_step`, we need to find if it's game over or not. To do that, we calculate the number of invaders on the screen at any given moment and see if it exceeds the maximum. On the other extreme, if it strikes your memory, you would remember that we did talk of how one can never win the game. There shall always be a minimum of four invaders persistent throughout.

There are several other methods present that deal with key presses, game pauses, game exits, crossover animation, etc. A few of them are shown in the image above and can be used if you see fit.

```
def recombine(pairs, gene_props, mutation_probability=0.1, effect=0.5):
    offspring = []
    for p1, p2 in pairs:
        children_genes = []
        for gen in p1.genes.keys():
            values = [p1.genes[gen], p2.genes[gen]]
            children_genes[gen] = random.uniform(min(values), max(values))
            if random.random() < mutation_probability:
                min_v = gene_props[gen]['min']
                max_v = gene_props[gen]['max']
                v = children_genes[gen]
                rv = random.choice([-1, 1]) * random.uniform(0, effect * (max_v - min_v))
                new_v_gauss = bound_value(random.gauss(v, (max_v - min_v) * effect), min_v, max_v)
                new_v = bound_value(v + rv, min_v, max_v)
                children_genes[gen] = new_v
            offspring.append(children_genes)
    return offspring
```

The crossover handled by recombine

RECOMBINE

Let's move on to the crux of the matter in hand, the bit of the code that actually deals with Genetic Algorithm. In order of its sequence, the first is crossover, for which we have a method called recombine.

The parameters that we are going to use for this function are the pairs of parents, the gene properties, the mutation probability and the effect.

We define a list to hold the offsprings. For each pair of parents we shall have a child gene formed which we are going to store in a dictionary. For each gene in the first parent's gene in the gene values list (the value part of the dictionary), we store both the values of the parent's gene. Now, for each of the child genes corresponding to the parent's gene, we use a distribution function of the random class to find a value to be allotted to the child gene with the minimum and maximum values as bounds for the distribution function being used.

If the random number generated is less than the mutation probability, we believe that it is time to mutate. We store the minimum and maximum values for the parent's genes. We already have the child's gene from the value computed above.

For mutation, we would basically replace a specific parameter in the gene by a random value in between its boundaries, with the help of the random functions. Having done

that, all we are left with is to append the children gene into the offspring gene. The next part deals with selection of the fittest individuals within the population.

SELECTING THE FITTEST

We begin by getting a summation of the fitness of all the individuals. We set up a separate list called `selected` which will contain the individuals that are selected. Next, we begin a `while` loop to until the length of the array is greater than a certain value. Inside the loop, we use a randomly generated uniformly distributed number with its limits as 0 and the sum calculated above, and this number is being subtracted by the evaluated individual's

```
def roulette_wheel(evaluated_population, k=10):
    sum_fitness = sum([v[1] for v in evaluated_population])
    selected = []
    while len(selected) < k:
        r = random.uniform(0, sum_fitness)
        for i in evaluated_population:
            r -= i[1]
            if r < 0:
                selected.append(i[0])
                break
    return selected
```

Selecting the fittest individuals

fitness' values. If the resultant value's function is less than zero, we consider adding that particular individual to the array selected that we have created earlier. This could have been done very simply by finding the top few individuals with the highest fitness functions but the interest of adding a random number distribution allows a broader scope of consideration keeping in mind the performance of the genetic individuals in the long run. What we see immediately may, only under the present circumstances, be perfect without any guarantee of a durable performance in the longer run. And this trend of adding a variety (be it in terms of a probabilistic distribution or something else) has always been used to improve the variety of the resultant individuals that will comprise the newer population. With this we have come to the end of this article, and at this point, the concept of a genetic algorithm should be quite clear. 